

Suma paralela de un vector con Free-Threaded Python: análisis de rendimiento

Institución: UNTDF

Carrera: Lic. en Sistemas, 5to año

Materia: Sistemas Paralelos

Docente: MsC. Federico González Brizzio

Abstract

El Global Interpreter Lock (GIL) de CPython ha sido históricamente la principal barrera para el paralelismo real con threads en Python. Este trabajo evalúa el rendimiento de cinco estrategias de concurrencia —secuencial, `ThreadPoolExecutor`, `ProcessPoolExecutor`, `multiprocessing.Pool` y `threading` manual— para la suma de un vector de complejidad c , utilizando Free-Threaded Python 3.14.3t (PEP 703) con el GIL deshabilitado. Se ejecutaron combinaciones de $p = 1, 2, 4, 8, 16$ workers y $c = 12, 10^6, 10^8$ en un equipo de 8 cores. Los resultados muestran que, sin GIL, los threads escalan genuinamente en tareas CPU-bound: `ThreadPoolExecutor` con $p = 4$ alcanza un speed-up máximo de 3.46x y eficiencia de 0.87 para $c = 10^8$. Se observa degradación de rendimiento más allá de $p = 4$, y que para cargas triviales ($c = 12$) el overhead de concurrencia domina completamente.

1. Introducción

Python es uno de los lenguajes más utilizados en computación científica, pero su modelo de concurrencia con threads ha estado históricamente limitado por el GIL (Global Interpreter Lock), un mutex que impide la ejecución simultánea de bytecode Python en múltiples threads. Esto fuerza a los desarrolladores a recurrir a procesos separados (`multiprocessing`) para lograr paralelismo real en tareas CPU-bound, con el costo asociado de serialización y comunicación inter-proceso.

La PEP 703 introduce Free-Threaded Python, una variante experimental de CPython que permite deshabilitar el GIL, habilitando paralelismo real con threads. Python 3.13+ ofrece builds con la opción `--disable-gil`, y a partir de 3.14 esta característica se consolida como opción estable.

El objetivo de este trabajo es evaluar empíricamente si Free-Threaded Python cumple su promesa: ¿pueden los threads escalar en tareas CPU-bound sin GIL? Para responder esta pregunta, se comparan cinco estrategias de concurrencia en un problema de suma paralela de un vector, variando la cantidad de workers y la complejidad del problema.

2. Metodología

2.1 Equipo

Propiedad	Valor
Cores (lógicos)	8
Python	3.14.3t (Free-Threaded)
GIL	Deshabilitado (PYTHON_GIL=0)
OS	Linux (Manjaro)

2.2 Algoritmos evaluados

1. **Secuencial:** iteración simple sobre el rango $[0, c)$, baseline para speed-up.
2. `concurrent.futures.ThreadPoolExecutor`: pool de threads de alto nivel con interfaz map.
3. `concurrent.futures.ProcessPoolExecutor`: pool de procesos con la misma interfaz.
4. `multiprocessing.Pool`: pool de procesos clásico de la biblioteca estándar.
5. **threading manual:** threads creados explícitamente con cola de tareas (Queue).

2.3 Parámetros experimentales

Parámetro	Valores
p (workers)	1, 2, 4, 8, 16
c (complejidad)	12, 10^6 , 10^8
Semilla	2026 (vector determinístico)

El vector no se materializa en memoria: cada elemento se genera bajo demanda mediante un generador congruencial lineal indexado por posición, garantizando reproducibilidad y evitando agotar RAM en $c = 10^8$.

2.4 Métricas

- **Speed-up:** $(S = T_{\text{secuencial}} / T_{\text{paralelo}})$
- **Eficiencia:** $(E = S / p)$

3. Resultados

3.1 $c = 12$

Carga trivial: 12 elementos. El overhead de creación de threads/procesos domina completamente.

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
secuencial	12	1	9.00e-06	1.0000	1.0000	8
concurrent.futures.ThreadPoolExecutor	12	1	7.55e-04	0.0119	0.0119	8
concurrent.futures.ThreadPoolExecutor	12	2	0.0011	0.0083	0.0042	8
concurrent.futures.ThreadPoolExecutor	12	4	0.0019	0.0048	0.0012	8
concurrent.futures.ThreadPoolExecutor	12	8	0.0019	0.0046	5.78e-04	8
concurrent.futures.ThreadPoolExecutor	12	16	0.0029	0.0031	1.93e-04	8
concurrent.futures.ProcessPoolExecutor	12	1	0.0958	9.40e-05	9.40e-05	8
concurrent.futures.ProcessPoolExecutor	12	2	0.0911	9.90e-05	4.90e-05	8
concurrent.futures.ProcessPoolExecutor	12	4	0.0975	9.20e-05	2.30e-05	8
concurrent.futures.ProcessPoolExecutor	12	8	0.0977	9.20e-05	1.20e-05	8

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
concurrent.futures.ProcessPoolExecutor	12	16	0.0975	9.20e-05	6.00e-06	8
multiprocessing.Pool	12	1	0.1077	8.40e-05	8.40e-05	8
multiprocessing.Pool	12	2	0.1044	8.60e-05	4.30e-05	8
multiprocessing.Pool	12	4	0.1100	8.20e-05	2.00e-05	8
multiprocessing.Pool	12	8	0.1218	7.40e-05	9.00e-06	8
multiprocessing.Pool	12	16	0.1318	6.80e-05	4.00e-06	8
threading	12	1	1.99e-04	0.0452	0.0452	8
threading	12	2	2.95e-04	0.0305	0.0153	8
threading	12	4	7.87e-04	0.0114	0.0029	8
threading	12	8	8.79e-04	0.0102	0.0013	8
threading	12	16	0.0012	0.0076	4.76e-04	8

3.2 $c = 10^6$

Carga media: un millón de elementos. Los threads comienzan a mostrar escalabilidad real.

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
secuencial	10^6	1	0.2792	1.0000	1.0000	8
concurrent.futures.ThreadPoolExecutor	10^6	1	0.2626	1.0630	1.0630	8
concurrent.futures.ThreadPoolExecutor	10^6	2	0.1329	2.0999	1.0500	8
concurrent.futures.ThreadPoolExecutor	10^6	4	0.0840	3.3221	0.8305	8
concurrent.futures.ThreadPoolExecutor	10^6	8	0.0889	3.1387	0.3923	8

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
concurrent.futures.ThreadPoolExecutor	10 ⁶	16	0.0946	2.9497	0.1844	8
concurrent.futures.ProcessPoolExecutor	10 ⁶	1	0.3571	0.7819	0.7819	8
concurrent.futures.ProcessPoolExecutor	10 ⁶	2	0.2234	1.2498	0.6249	8
concurrent.futures.ProcessPoolExecutor	10 ⁶	4	0.1771	1.5768	0.3942	8
concurrent.futures.ProcessPoolExecutor	10 ⁶	8	0.1864	1.4980	0.1873	8
concurrent.futures.ProcessPoolExecutor	10 ⁶	16	0.1999	1.3965	0.0873	8
multiprocessing.Pool	10 ⁶	1	0.3688	0.7569	0.7569	8
multiprocessing.Pool	10 ⁶	2	0.2492	1.1202	0.5601	8
multiprocessing.Pool	10 ⁶	4	0.2622	1.0649	0.2662	8
multiprocessing.Pool	10 ⁶	8	0.2318	1.2045	0.1506	8
multiprocessing.Pool	10 ⁶	16	0.2761	1.0111	0.0632	8
threading	10 ⁶	1	0.2551	1.0945	1.0945	8
threading	10 ⁶	2	0.1386	2.0136	1.0068	8
threading	10 ⁶	4	0.1170	2.3865	0.5966	8
threading	10 ⁶	8	0.0903	3.0901	0.3863	8
threading	10 ⁶	16	0.0929	3.0053	0.1878	8

3.3 c = 10⁸

Carga pesada: cien millones de elementos. Paralelismo real visible con todas las estrategias.

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
secuencial	10 ⁸	1	27.08	1.0000	1.0000	8
concurrent.futures.ThreadPoolExecutor	10 ⁸	1	26.49	1.0221	1.0221	8
concurrent.futures.ThreadPoolExecutor	10 ⁸	2	13.92	1.9447	0.9724	8
concurrent.futures.ThreadPoolExecutor	10 ⁸	4	7.82	3.4608	0.8652	8

Algoritmo	c	p	Tiempo (s)	Speed-up	Eficiencia	Cores
concurrent.futures.ThreadPoolExecutor	10^8	8	8.71	3.1097	0.3887	8
concurrent.futures.ThreadPoolExecutor	10^8	16	8.89	3.0459	0.1904	8
concurrent.futures.ProcessPoolExecutor	10^8	1	27.87	0.9715	0.9715	8
concurrent.futures.ProcessPoolExecutor	10^8	2	15.17	1.7853	0.8927	8
concurrent.futures.ProcessPoolExecutor	10^8	4	8.71	3.1081	0.7770	8
concurrent.futures.ProcessPoolExecutor	10^8	8	9.26	2.9240	0.3655	8
concurrent.futures.ProcessPoolExecutor	10^8	16	9.39	2.8828	0.1802	8
multiprocessing.Pool	10^8	1	28.18	0.9609	0.9609	8
multiprocessing.Pool	10^8	2	14.30	1.8932	0.9466	8
multiprocessing.Pool	10^8	4	8.23	3.2918	0.8230	8
multiprocessing.Pool	10^8	8	9.11	2.9728	0.3716	8
multiprocessing.Pool	10^8	16	9.18	2.9488	0.1843	8
threading	10^8	1	27.50	0.9844	0.9844	8
threading	10^8	2	14.28	1.8965	0.9482	8
threading	10^8	4	8.15	3.3240	0.8310	8
threading	10^8	8	8.70	3.1110	0.3889	8
threading	10^8	16	8.96	3.0205	0.1888	8

4. Discusión

4.1 Carga trivial (c = 12): el overhead domina

Con solo 12 elementos, el trabajo útil (sumar 12 valores) toma $\sim 9 \mu\text{s}$. Cualquier mecanismo de concurrencia introduce overhead que supera en órdenes de magnitud el cómputo real:

- **Procesos** (ProcessPoolExecutor, multiprocessing.Pool): $\sim 100 \text{ ms}$ de overhead por fork/spawn, resultando en speed-ups del orden de 10^{-4} – 10^{-5} .

- **Threads** (ThreadPoolExecutor, threading): ~0.2–3 ms de overhead, mejor que procesos pero aún 2 órdenes de magnitud más lentos que el secuencial.

El secuencial es imbatible aquí. Esto confirma que la paralelización solo tiene sentido cuando la carga computacional justifica el costo de coordinación.

4.2 Carga media ($c = 10^6$): threads escalan, procesos luchan

Con un millón de elementos, el trabajo útil toma ~0.28 s. Aquí se manifiesta la ventaja de Free-Threaded Python:

- **Threads sin GIL** escalan genuinamente: ThreadPoolExecutor alcanza 3.32x con $p = 4$, y threading manual logra 3.09x con $p = 8$.
- **Procesos** escalan poco: ProcessPoolExecutor llega a 1.58x con $p = 4$. El overhead de serialización IPC limita la ganancia.
- **Eficiencia máxima:** threading con $p = 1$ alcanza 1.09, ligeramente superior al secuencial por variabilidad de medición.

4.3 Carga pesada ($c = 10^8$): paralelismo real

Con cien millones de elementos (~27 s secuencial), todas las estrategias muestran escalabilidad:

- **Mayor speed-up:** ThreadPoolExecutor con $p = 4 \rightarrow 3.46x$.
- **Todas las estrategias** convergen a ~3.0–3.5x con $p = 4$, evidenciando que sin GIL, threads y procesos tienen rendimiento comparable.
- **Degradación con $p > 4$:** al pasar a $p = 8$, el speed-up cae a ~3.1x; con $p = 16$ baja a ~3.0x. Esto se explica porque el equipo tiene 8 cores lógicos (probablemente 4 físicos + hyperthreading), y con más de 4 threads el scheduling y la contención de caché introducen overhead.

4.4 Threads vs. procesos sin GIL

Un hallazgo notable es que sin GIL, **threads y procesos rinden casi idéntico** para esta tarea CPU-bound pura. Esto contrasta radicalmente con CPython estándar (con GIL), donde los threads no pueden paralelizar cómputo CPU-bound y solo los procesos escalan. Free-Threaded Python elimina esa asimetría.

Los threads incluso tienen una ligera ventaja sobre procesos por evitar el overhead de fork/spawn y serialización de datos.